IŞIK UNIVERSITY FACULTY OF ADMINISTRATIVE ECONOMICS AND SOCIAL SCIENCES
Department of Management Information Systems

AI-FASHION

WARDROBE ASSISTANT USING COMPUTER VISION AND COLOR HARMONY

Mohamed Hatem Salah Mostafa Elasar

A Bachelor' s Thesis
in the Department of Management Information Systems
Faculty of Economics, Administrative and Social Sciences

Supervisor: Prof. Şahin Aydın

AI-Fashion

A Project Presented

by

Mohamed Hatem Salah Mostafa Elasar

22MISY1059

to

The Department of Management Information Systems

in partial fulfillment of the requirements

for the degree of

Bachelor of Science

in the field of

Management Information Systems

Işık University

İstanbul, Turkey

January 2026

Dedicated to the Mentors that have inspired and motivated me to pursue great things in life..

Abstract

AI-Fashion is a smartphone application that manages the wardrobe automatically and offers personalized outfits suggestions guided by the principles of the color theory.

The system uses a multi-stage processing pipeline, which is the combination of computer vision, machine learning, and color science. Photos of clothes uploaded are processed using U2Net to remove the background, Google Gemini Vision API to classify the clothes, and K-Means clustering to extract the dominant color in the HSL color space.

The recommendation engine follows the “Goldilocks Principle” of Gray et al. (2014), who discovered that the ideal fashionability is a moderate coordination of colors. The algorithm employs weighted scoring: color harmony (60%), weather suitability (20%), and occasion suitability (20%): the combinations are evaluated using the complementary, analogous, triadic, and split-complementary color strategies.

The technical stack contains Python-FastAPI backend, Flutter, and Provider to create cross-platform mobile applications, SQLite to store data locally, and Open-Meteo API to integrate weather.

Acknowledgements

I want to take a moment to thank all the people who helped make this thesis project happen.

To start with, I would like to give a tremendous credit to my supervisors, Şahin Aydın and Deniz Altıntaş. Their guidance, feedback, and constant support in this study helped its development.

I also appreciate the Faculty of Economic, Administrative and Social Sciences of Isik University that provided me with the academic environment and facilities that I required to conduct this research. All the knowledge I acquired in my studies of Management Information Systems was the foundation of this project.

A special mention is made to the open-source community to develop and support the tools and libraries that enabled all of this, such as FastAPI, Flutter, rembg, and scikit-learn. Access to the Gemini Vision API by Google was also necessary to make the clothing detection piece functional.

I would also like to note the names of the researchers whose work on the color theory and fashion psychology contributed to the theoretical part of the application. Specifically, one of the significant influences was Gray et al. and their revolutionary work on the Goldilocks Principle in fashion.

And finally, though most certainly not the least, I am extremely grateful to my family. They motivated me by their love, support and tolerance even when I was fully in my academic journey. They have always inspired me.

Contents

Chapter 1 Overview

This part marks the starting point of the thesis. It introduces the research problem, explains why it is relevant, and frames the overall structure of the work. The chapters in this part provide the conceptual and methodological groundwork for the analysis presented later.

1.1 Introduction

The intersection of artificial intelligence and fashion technology provides solutions to daily problems in a practical way. The growing number of wardrobes and the variety of fashions have led to the phenomenon of decision fatigue as individuals struggle to choose a daily outfit (Schwartz, 2004). It is solved by AI-Fashion using an intelligent mobile application that automatizes wardrobe management and provides personalized outfit selection on the basis of scientific color harmony principles.

The major innovation of this project is the application of the "Goldilocks Principle" proposed by Gray et al. (2014), whose study proves that moderate color coordination brings about maximum fashionability, neither too matched nor too clashing. AI-Fashion provides advice based on scientifically determined fashion aesthetic by converting these empirical data points into algorithmic form.

The app uses a multi-stage processing pipeline, which combines various AI applications: Google Gemini Vision API, clothing detection (Google, 2024), the U2Net deep learning framework, background removal (Qin et al., 2020) and K-Means clustering on the HSL color space, which is used to analyze colors to support complementary, analogous, and triadic harmonies (Itten, 1961).

The technical architecture is based on modern software engineering, and it has a Python FastAPI back-end and Flutter cross-platform mobile deployment.

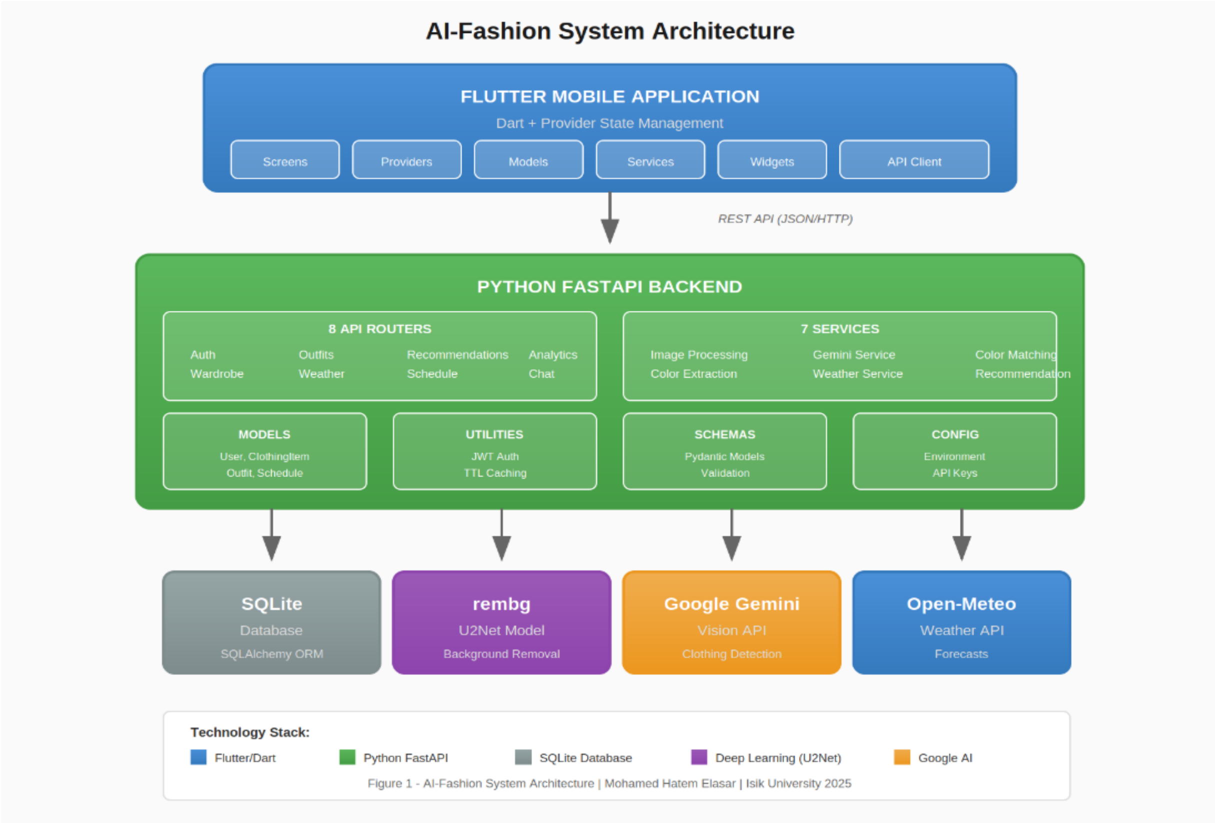

Figure 1. High-level application architecture of the AI-Fashion system

1.2 Motivation

The idea behind AI-Fashion is shaped by one of the most common daily problems: what to wear. The studies indicate that the average person spends about 17 minutes a day deciding what to wear, which amounts to more than 100 hours a year (ClosetMaid, 2017). This kind of decision-making process usually results into needless stress and displeasure (Schwartz, 2004).

There are two types of current solutions in the market: basic inventory applications with no smart suggestions, or costly styling options that need human stylists. There is a considerable gap towards an available, AI-based solution integrating wardrobe management and smart outfit recommendations.

Color coordination is one of the problems that most individuals find challenging. The Goldilocks Principle (Gray et al., 2014) is a recent scientific theory that demonstrates that moderate color matching attracts the highest fashionability scores. Also, the environmental footprint of the fashion industry inspires this project-by assisting users to stretch existing collections through smart matching, AI-Fashion will promote sustainable consumption habits (Ellen MacArthur Foundation, 2017).

New technologies in mobile computing and cloud-based AI services enable the deployment of advanced machine learning models in consumer applications now (Dean et al., 2018).

1.3 Contributions

This thesis makes several significant contributions:

Novel Color Harmony-First Algorithm: A recommendation algorithm based on aesthetic color relations where color harmony is rated as 60 %, weather 20 % and occasion 20 percent. This strategy is based on the empirical results of Gray et al. (2014).

Integrated AI Processing Pipeline: A holistic pipeline that consists of U2Net to remove the background, Gemini Vision API to categorize clothes, and K-Means to cluster colors, to achieve accurate analysis at the same time keeping the processing speed of mobile standards.

Algorithmic Implementation of Fashion Research: The Goldilocks Principle as a Computer-Assisted Recommendation System: The object is to translate the academic research in fashion psychology into software programs.

Comprehensive Mobile Application: A fully functional application with scheduling outfits weekly, wardrobe insights, and a chatbot that understands its users and provides a viable implementation of FastAPI, Flutter, SQLite, and AI solutions in the cloud.

HSL-Based Color Analysis Framework: An analysis framework of clothing colors on HSL color space, with the ability to make detailed judgments of complementary, analogous, triadic, and split-complementary harmonies.

1.4 Outline of the Project

This thesis is structured with seven chapters that follow the steps of the research, design, implementation, and evaluation of the AI-Fashion application.

Chapter 1 (Overview) has provided the background, rationale and contribution of this study. The chapter has defined the issue of decision fatigue in outfit selection and presented the color harmony first approach as the main innovation.

Chapter 2 (Literature Review) is a comprehensive analysis of the existing literature in three main points: fashion recommendation systems and their development, color theory and harmony principles to fashion, and the artificial intelligence technologies applied to the application. This chapter presents the theoretical basis that the project is based on and the gaps in the existing solutions that AI-Fashion closes.

Chapter 3 (System Architecture and Design) outlines the system architecture, including the fine details of the backend (FastAPI), the frontend design (Flutter), and the database (SQLite) structure. The chapter describes the technical choices and design patterns of the entire project, such as RESTful API design and state management strategy.

Chapter 4 (AI and Machine Learning Methods) is dedicated to the artificial intelligence and machine learning methods applied in the application. These contain descriptions of the U2Net model of background removal (Qin et al., 2020), the Gemini Vision API of clothing detection (Google, 2024), K-Means clustering of colors, and the HSL based color harmony analysis algorithms based on research in color theory (Itten, 1961).

Chapter 5 (Implementation) is the practical implementation area, which includes displaying the code structure, API endpoints, mobile application screens, and the integration of different components. This chapter provides the idea of how the theoretical concepts and design decisions were transformed into working software.

Chapter 6 (Results and Discussion) reports the findings of system evaluation and discusses findings. This involves accuracy measurements on clothing detection, color extraction performance, recommendation quality based on correlation with human preference and user experience measurements.

In Chapter 7 (Conclusion), the main project outcomes are summarized, limitations to the current implementation are addressed, and suggestions to further research and advancement in AI assisted fashion technology are made.

Chapter 2 Research Background

This chapter provides a critical discussion of the literature applicable in developing AI-Fashion, an intelligent outfit recommendation system. The review is divided into three major parts that correspond to the most prominent areas that meet in this study: artificial intelligence in fashion technology, computer vision in clothing recognition, and color theory in fashion aesthetics..

2.1 Research Context

Fashion technology is a particularly rapidly expanding field due to the development of machine learning, computer vision, and mobile computing in recent years (Cheng et al., 2021).

Scholars and practitioners in the industry have discussed a range of ways to automate fashion related activities, with virtual try on systems and outfit recommendation engines. This landscape is critical to understanding in positioning AI-Fashion as a research on fashion technology.

Although it is an ancient practice in the field of art, the color theory was only empirically studied in the framework of fashion in recent years (Gray et al., 2014). The theoretical basis of the AI-Fashion recommendation algorithm is the scientific study of color balance and its use in the coordination of clothing.

The chapter examines the classical color theory and current studies that guide the system in assessing outfits. The review also addresses the actual AI technologies applied in the application, such as deep learning models to process images and clustering algorithms to extract colors. This chapter provides the technical background of the implementation decisions presented in the following chapters by analyzing the state of the art in these fields

2.2 AI in Fashion Technology

Artificial intelligence in fashion is a fast emerging subdiscipline of computer science, design, and consumer behavior studies. The initial efforts were made in recognition and retrieval of clothes, and scholars created systems that could recognize garment types based on pictures (Chen et al., 2012). DeepFashion data, proposed by Liu et al. (2016), was an important breakthrough since it contained more than 800,000 annotated images and allowed the creation of sophisticated models.

Fashion recommendation systems have developed beyond collaborative filtering (Sarwar et al., 2001) to deep learning models. Content-based techniques examine visual attributes and text descriptions in order to find similar objects (McAuley et al., 2015). Recent studies make use of attention mechanisms and transformer architectures to capture complex fashion item relationships (Hou et al., 2019).

Outfit recommendation is a different set of challenges compared to individual item recommendation. Han et al. (2017) proposed the use of the bidirectional LSTM networks in learning the compatibility of fashion items and showed that outfit compatibility could be trained using user-curated collections. Virtual try-on technologies, including VITON (Han et al., 2018), expanded the use of AI to show how clothing would look on others.

Fashion research that is context-aware now encompasses context-aware weather-aware recommendation (Chen et al., 2019), social context like cultural norms and occasion-related fashion (Lin et al., 2020). AI-Fashion builds up on the concept by incorporating not only weather predictions but the type of occasion during their suggestions. However, most fashion AI frameworks are insensitive to basic colour theory, and colour is an arbitrary learned attribute that is not associated with domains (Hsiao and Grauman, 2018). To solve this, AI-Fashion operates under a colour harmony-first approach based on the recognised principles of coordination.

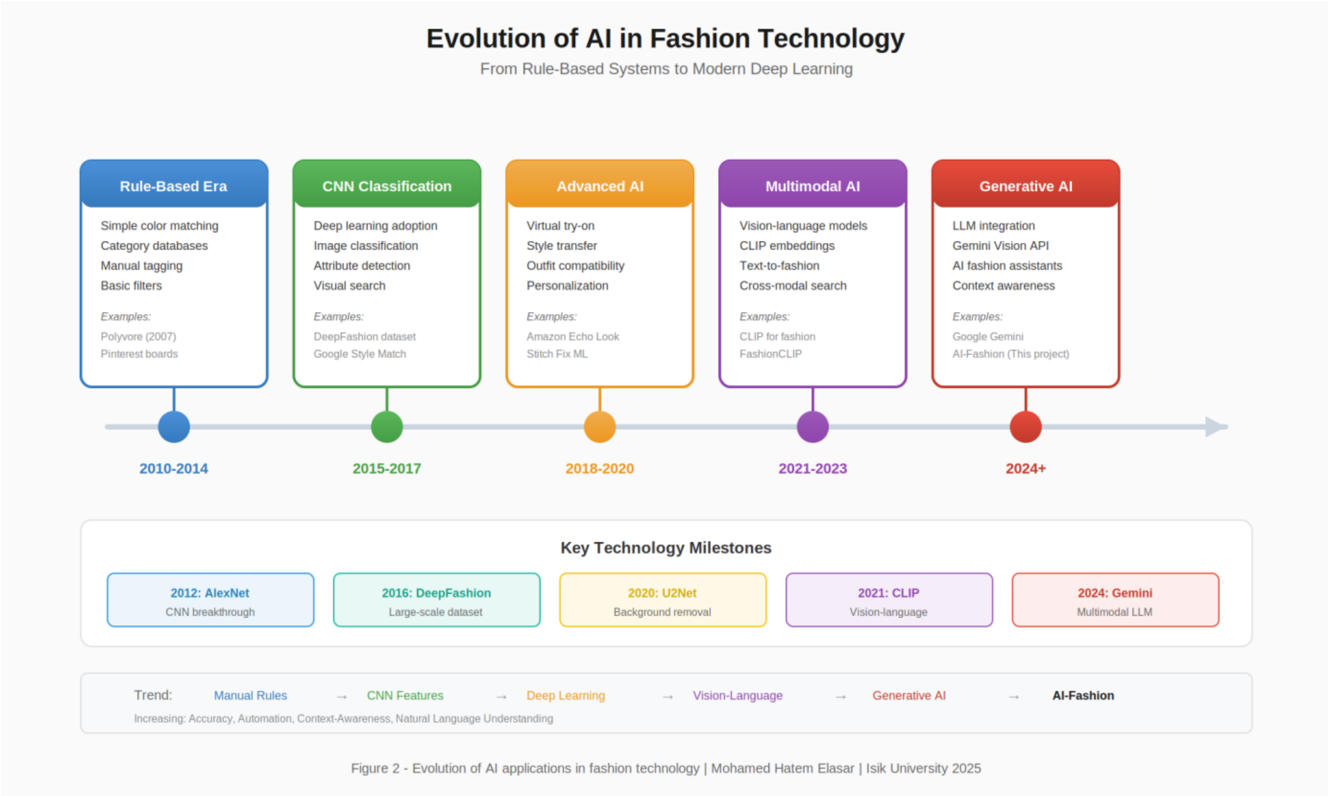

Figure 2. Evolution of AI in fashion technology

2.3 Computer Vision for Clothing Recognition

Clothing recognition by computer vision has been actively developed, moving past the older feature-based methods to more complex deep learning methods. Early ones were based on features of handcrafting like SIFT (Lowe, 2004) and HOG (Dalal and Triggs, 2005). Although they worked with limited datasets, they failed in practice with real world variability in pose, light and occlusion.

The revolution in this field was made by deep convolutional neural networks. Krizhevsky et al. (2012) proved breakthrough performance with AlexNet, then more advanced architectures were presented, such as VGGNet (Simonyan and Zisserman, 2014) and ResNet (He et al., 2016). Specialized applications were introduced by DeepFashion dataset (Liu et al., 2016), which allows training special models that can recognize clothes and predict their attributes.

Background elimination is one of the key preprocessing stages of fashion image. The U2Net design (Qin et al., 2020) proved the highest performance in the salient object detection with its nested U-structure framework that is efficient in both local and global detail feature. This technology has been democratized by the rembg library (Gatis, 2021) so the developers can use it to implement advanced segmentation without the need to know deep learning.

Clothing recognition is a new development with vision-language models. Other systems such as the Gemini by Google (Google, 2024) integrate both the vision and language in a manner that they can explain types of clothing and their features in their context, and commonly they give us the structured natural-language outputs (including item description and confidence), not just fixed classes.

To analyse colour, K-means can be used to extract dominating colours in clothing pictures (Lloyd, 1982). The colour space is important: HSL gives spacing between colour and brightness and is easier to use to understand colour relationships than RGB (Fairchild, 2013; Itten, 1961).

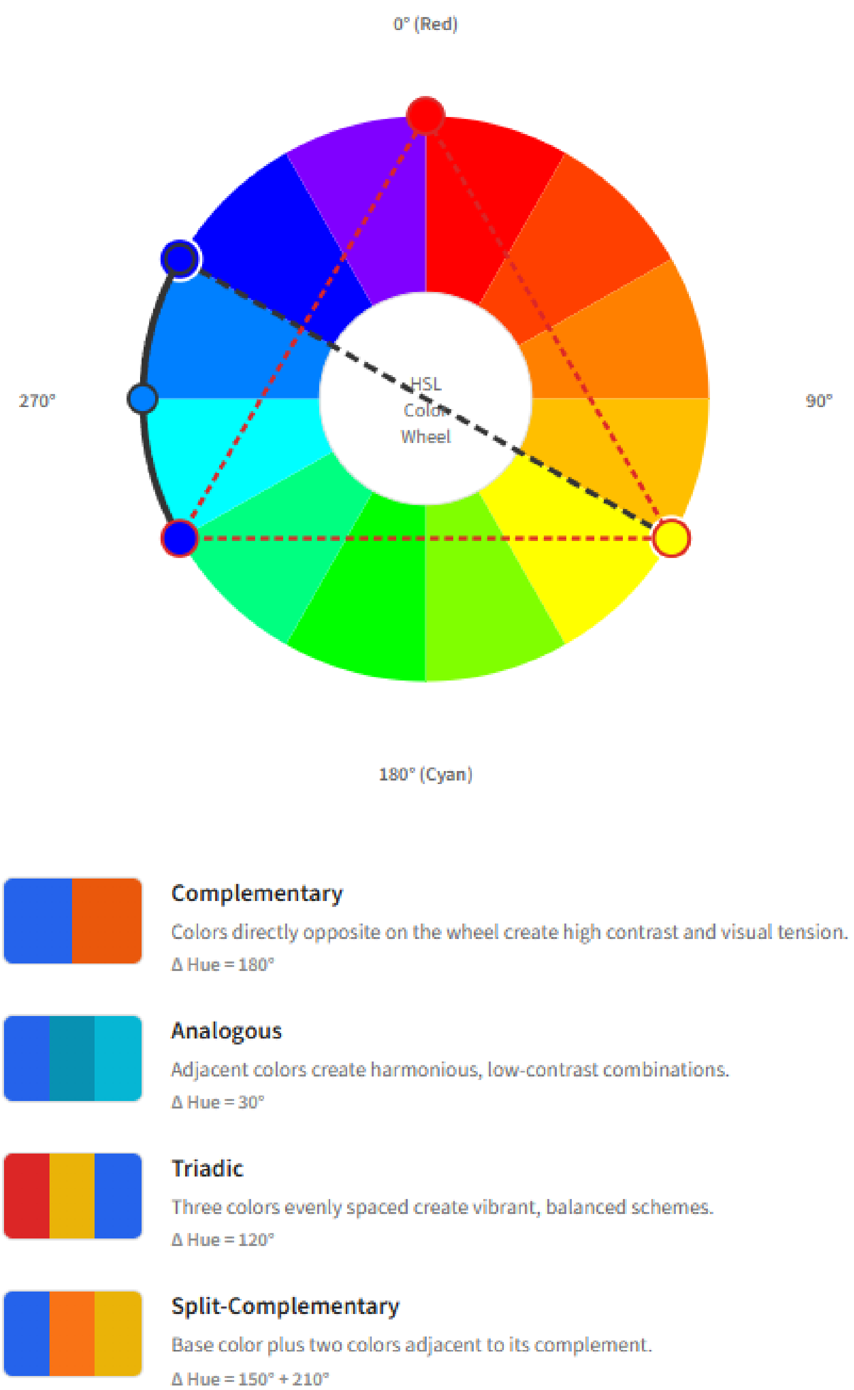

Figure 3. Color wheel with four harmony strategies

Chapter 3 System Architecture and Design

This chapter describes the architecture of the AI-Fashion application in detail, including the system elements and their interdependence with others. It is based on a client server model where the Python FastAPI server handles AI processing and data management, and a Flutter mobile application is used as the user interface.

3.1 System Overview

This separation of concerns enables each part to be scaled and maintained separately and at the same time provide a responsive user experience. Its architecture was built based on several concepts: modularity to facilitate easier future testing and future improvements, efficiency to support real time mobile interactions, and extensibility to enable future feature additions.

Figure 4 is the high level system architecture and it reflects how data flows between components.

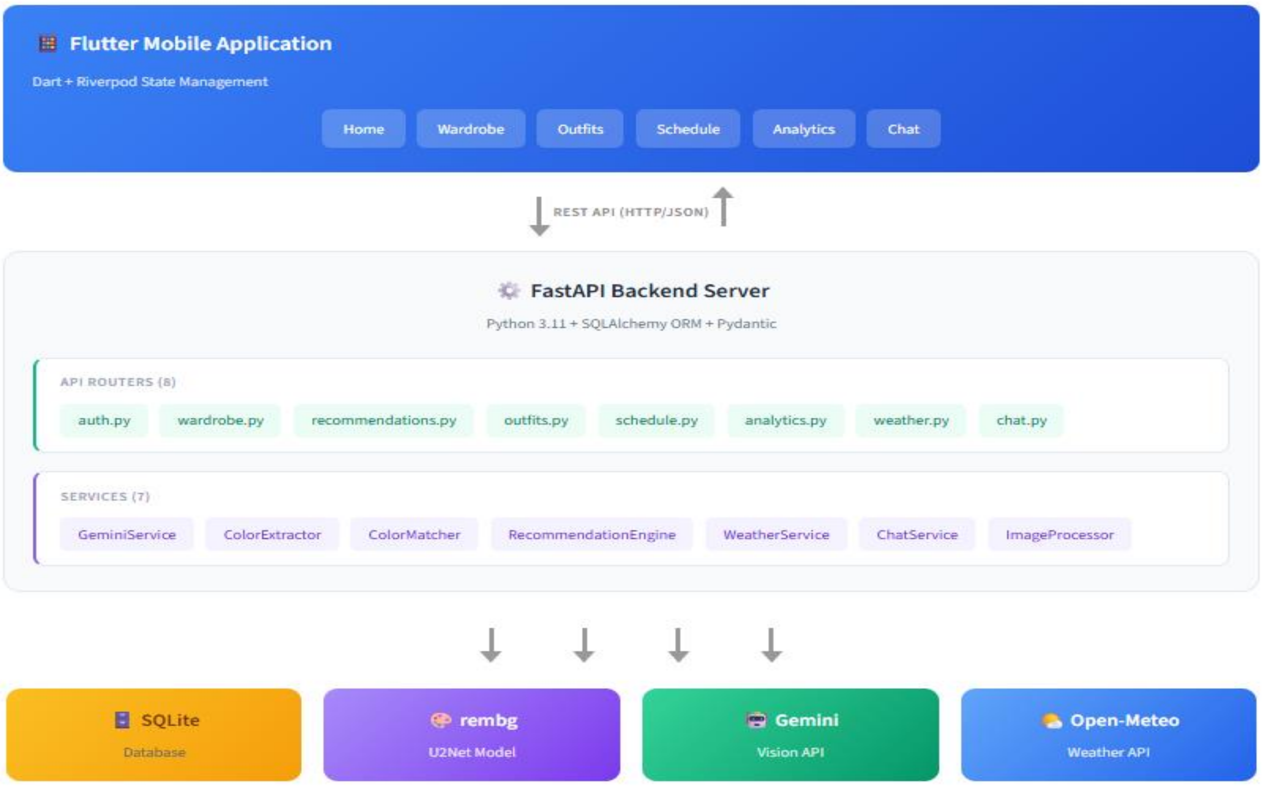

Figure 4. System component diagram

3.2 Backend Architecture

The server is developed based on FastAPI, a recent Python web framework that is delivered with respect to its high-performance and automatic API documentation features (FastAPI, 2024). FastAPI was selected among such alternatives as Flask and Django due to the support of asynchronous work, automatic request validation in Pydantic models, and automatic OpenAPI documentation generation.

The back-end is divided into eight major routers, each of which has a particular functionality: Authentication (user registration and JWT token management), Wardrobe (CRUD operations with clothes), Weather (integration with external APIs), Recommendations (generation of outfits), Outfits (saving outfits), Scheduling (planning of the week), Analytics (statistics of a wardrobe), and Chat (conversations with an AI assistant). It is a modular framework, which adheres to the Single Responsibility Principle and makes the codebase maintainable and testable.

Service layer contains the business logic and AI processing unit. Among the major services are the Gemini Service to detect clothing, Color Extractor to recognize the prevalent colors, Color Matcher to analyze harmony and Recommendation Engine to generate outfits. The services are configured as singleton to maximize resource utilization and make the state the same in requests.

The externality integrations are Google Gemini Vision API to analyze clothes and Open Meteo to retrieve weather data. The Gemini API manages clothing type recognition, layer recognition, and attribute recognition. The weather information renders context conscious suggestions possible by modifying suggestions, depending on the weather, precipitation, and so on.

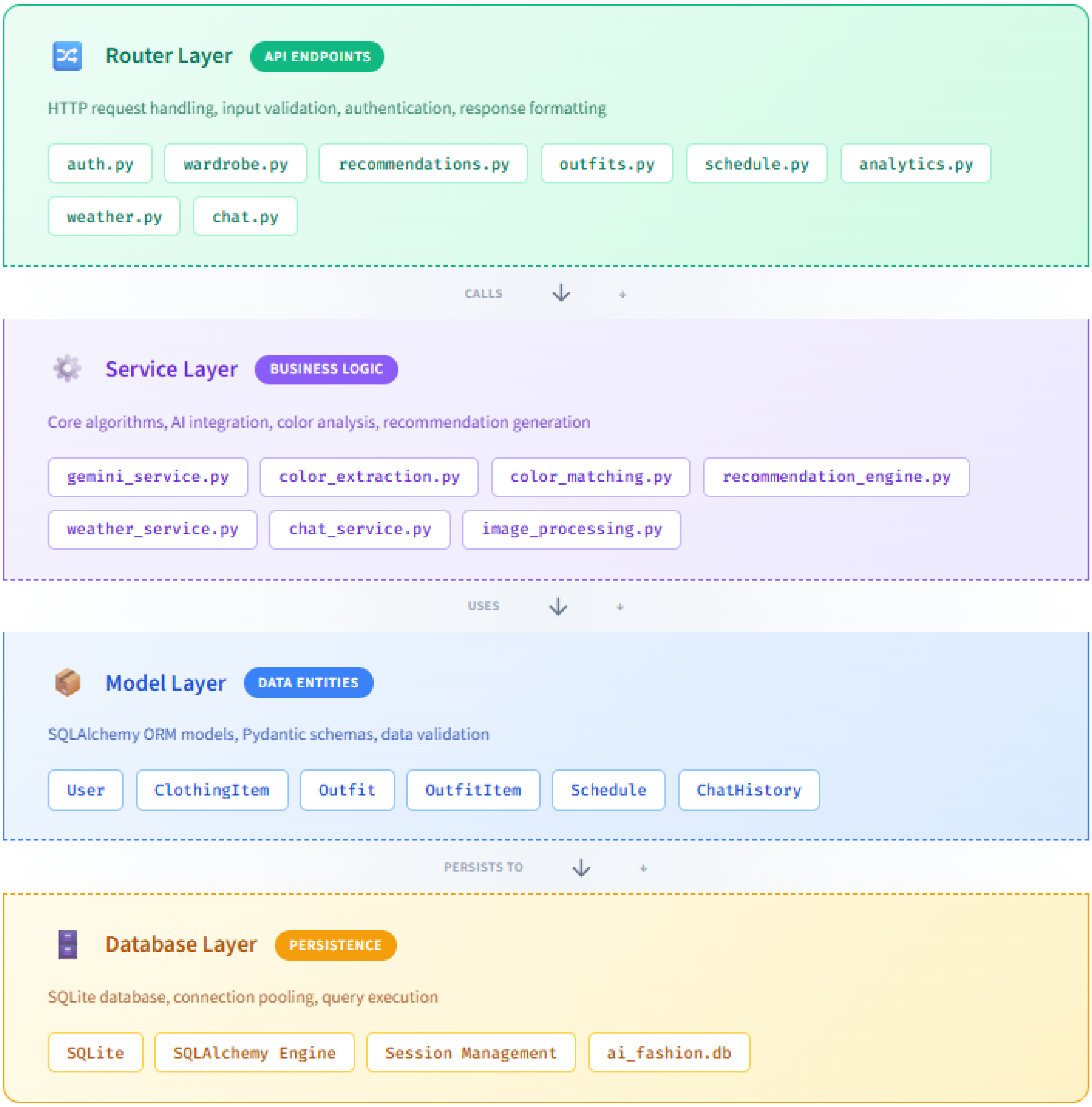

Figure 5. Backend layered architecture

3.3 Frontend Architecture

The mobile app will be developed with Flutter, which is a cross platform UI framework created by Google and allows it to run on both iOS and Android using a single source. Flutter was selected due to its high performance nature, opulent widget library and excellent developer tooling. The frame is compiled to native code, bypassing the performance overhead of the bridges of other frameworks.

State is managed with Provider, a data binding reactive caching framework which provides compile time safety and it is also testable (Provider, 2024). The dependency injection and state sharing across the widget tree is effectively supported by Provider system. It applies a combination of StateNotifierProvider of complex state and FutureProvider of data asynonymous loading.

The UI is based on Material Design 3 and uses a custom color pallet that focuses on fashion forward design. The application is built on a bottom menu that has five primary areas: Home (daily recommendations), Wardrobe (item management), Outfits (saving combinations), Schedule (planning weekly), and Profile (settings and analytics). Every part of the site is created as an independent module and has its providers and widgets.

Frontend involves image management. The program relies on the image picker package to access the camera and gallery with automatic compression which reduces upload time and storage needs. Stored network images enhance speed in showing wardrobe products, whilst hero animations allow a smooth transition between list and detail screen.

3.4 Database Design

Data persistence is controlled by SQLite, the weightless relational database which is particularly effective in mobile and embedded software. SQLAlchemy ORM is used by the backend in database operations to offer an abstraction layer that simplifies the query and provides type safety (SQLAlchemy, 2024). The database structure consists of five major tables: Users, ClothingItems, outfits, Schedules and ChatHistory.

Users table contains authentication detail and profile data, passwords are hashed with bcrypt because hashing provides security enhancements. The table ClothingItems contains information about wardrobes such as image pathways, types of clothing identified, types of layers, color data (primary and secondary hexes), and attributes in JSON format. The foreign key relationships between clothing items and users allow multi user support.

The Outfits table forms a many to many union, with ClothingItems via a junction table so we have the flexibility of composition of outfits. Outfit records have such metadata as type of occasion, date of creation and name assigned by the user. The Schedules table routes the outfits to particular dates, which drives the weekly planning option.

The primary and secondary colors extracted off each item are stored as hexadecimal strings of color data. The format manages to make storage efficient and still covers the entire palette of colors. Conversion to HSL required to perform harmonic analysis occurs at run time making analysis algorithms flexible without the need to migrate the database.

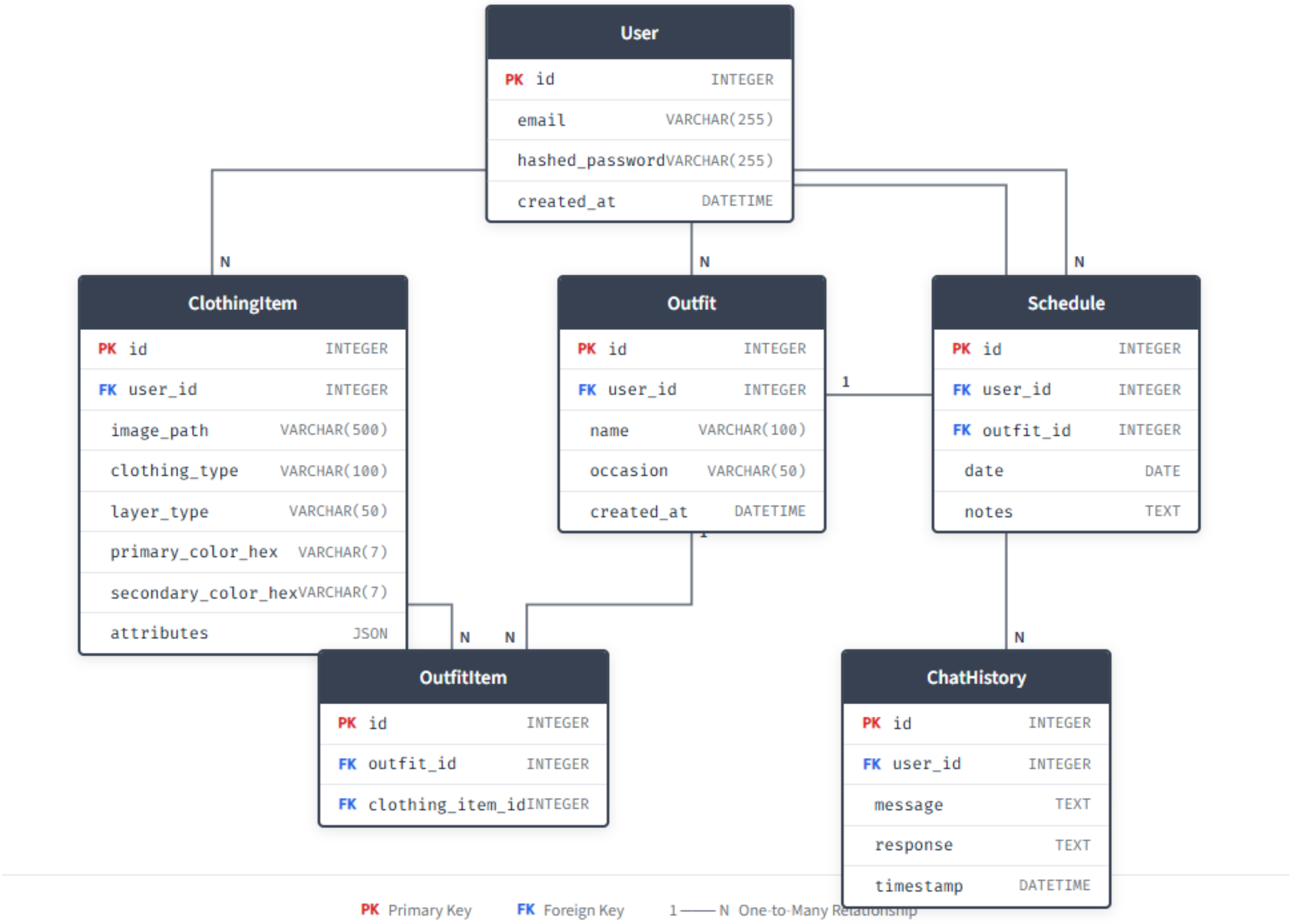

Figure 6. Database entity-relationship diagram

Color Harmony Algorithms

The color harmony analysis system applies the principles of classical color theory to practice, making the works of Itten (1961) and Munsell (1905) algorithmic. Colors are broken down in HSL (Hue, Saturation, Lightness) color space, which isolates chromatic data and the data of brightness and lets it be more intuitively analyzed to understand color relationships.

ColorMatcher service recognizes various sets of color relationships. Complementary colors refer to colors whose color values differ by approximately 180 degrees on the color wheel forming high contrasts. Parallel colors consist of colors of 30 degrees apart, and they create low contrast matches. Triadic relationship utilizes three color spaced at 120 degrees whereas split complementary employs one base color and the other two adjacent to it.

Fashion analysis requires the neutral color recognition as neutrals (black, white, gray, navy, beige, brown) match pretty much any chromatic colour. The system categorizes color as neutral in terms of saturation thresholds, and even the hue saturation lightness ranges that match up with fashion neutrals. This segmentation renders the strategy of the neutral plus accent feasible, which is the basis of numerous successful outfits.

The principle of the Goldilocks Principle has been applied to punish over-similarity (matchy matchy) and under-similarity (clashing) and reward moderate coordination. This is achieved by a scoring mechanism which assesses the difference in hues with the highest scores lying in a range that implies similar but distinct colors (Gray et al., 2014).

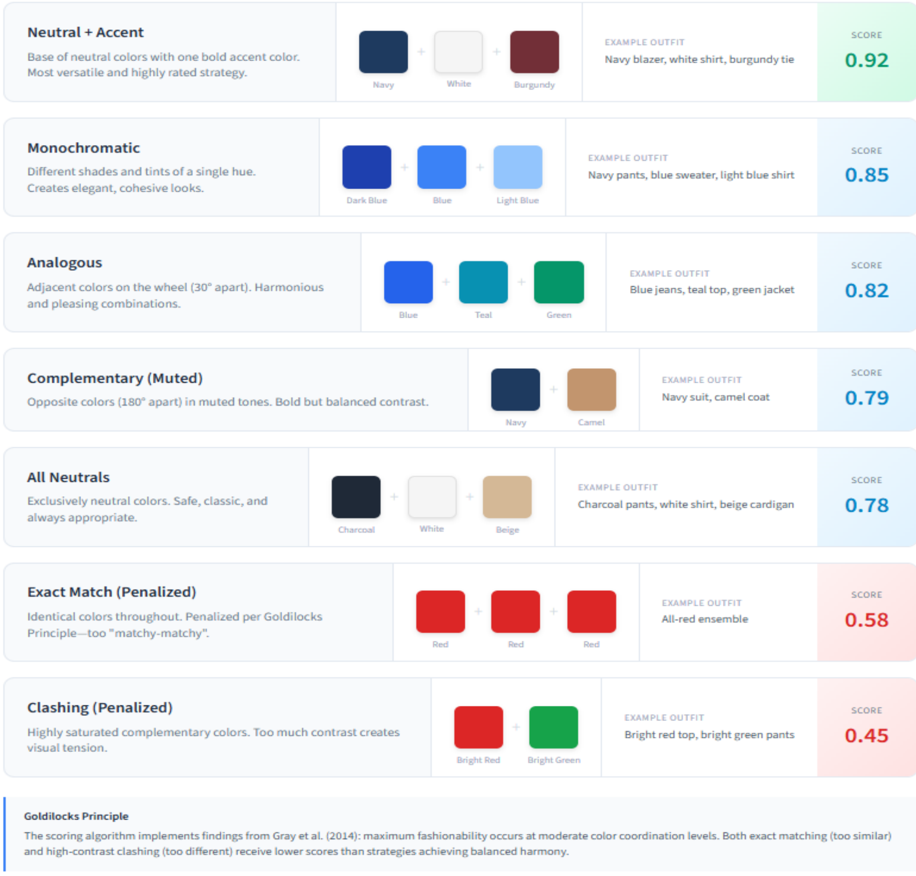

Figure 7. Color harmony scoring strategies

Recommendation Engine

The RecommendationEngine service is in charge of the process of generating outfits, which combines the COA analysis with the rests of the weather and occasion. A scoring system of the engine will be weighted: 60 per cent. on color harmony, 20 per cent. on weather-fitness, and 20 per cent. on occasion-fitness. Such weighting is informed by the research results that indicate the primary determinant of the appearance of a good outfit, that color coordination (Gray et al., 2014; Li et al., 2025).

Generation or outfit generation begins with the selection of a top item (usually a top or bottom). The engine proceeds to select complementary items to the remaining layers (top, bottom, outerwear, footwear, socks) according to the compatibility of colors with the already picked pieces. The candidate items are scored against the cumulative color palette and the highest scoring item is selected in each layer.

Weather integration changes according to weather based on temperature, rainfall and conditions. The system establishes requirements of layers in various weather conditions. Cold weather initiates outerwear suggestions whereas warm weather excludes heavy clothing. The Open Meteo API provides weather information according to user-location and buffers them to minimize API calls.

Filtering filtering ensures that recommendations are socially appropriate. There are six occasions: Casual, Formal/Business, Sport/Athletic, Party/Night Out, Date, and Home/Loungewear. Every event is conducive and forbids qualities (such as formal events favor elegant, and discourage sporty), and the degree of formality which will dictate colors suggestions.

To provide some choice, the engine generates various outfit choices (several by default) so that there is variety thereupon. Composite score ranks results with the highest scoring outfit indicated as the main recommendation. Together with the strategy of color, as well as harmony relations, each recommendation is accompanied by a natural language description.

Chapter 4 AI and Machine Learning Methods

This chapter delves in the AI and Machine learning technology that was theoretically and digitally applied throughout the AI-Fashion Application.

4.1 Methods Introduction

The system combines various AI applications in a processing chain that processes raw clothing images and converts them into analysable data that can be utilized in the creation of recommendations. Three steps of removing the background and detecting and classifying clothes with Google Gemini Vision API and color extraction with K-Means clustering are the key stages in the pipeline.

All technologies were selected according to their performance features, integration ease, and compatibility with mobile application deployment. By integrating these methods, the uploaded clothing images of users can be analyzed in a punctual way and extract the data to be utilized in intelligent outfit suggestions. The complete pipeline of processing picture is illustrated in figure 8.

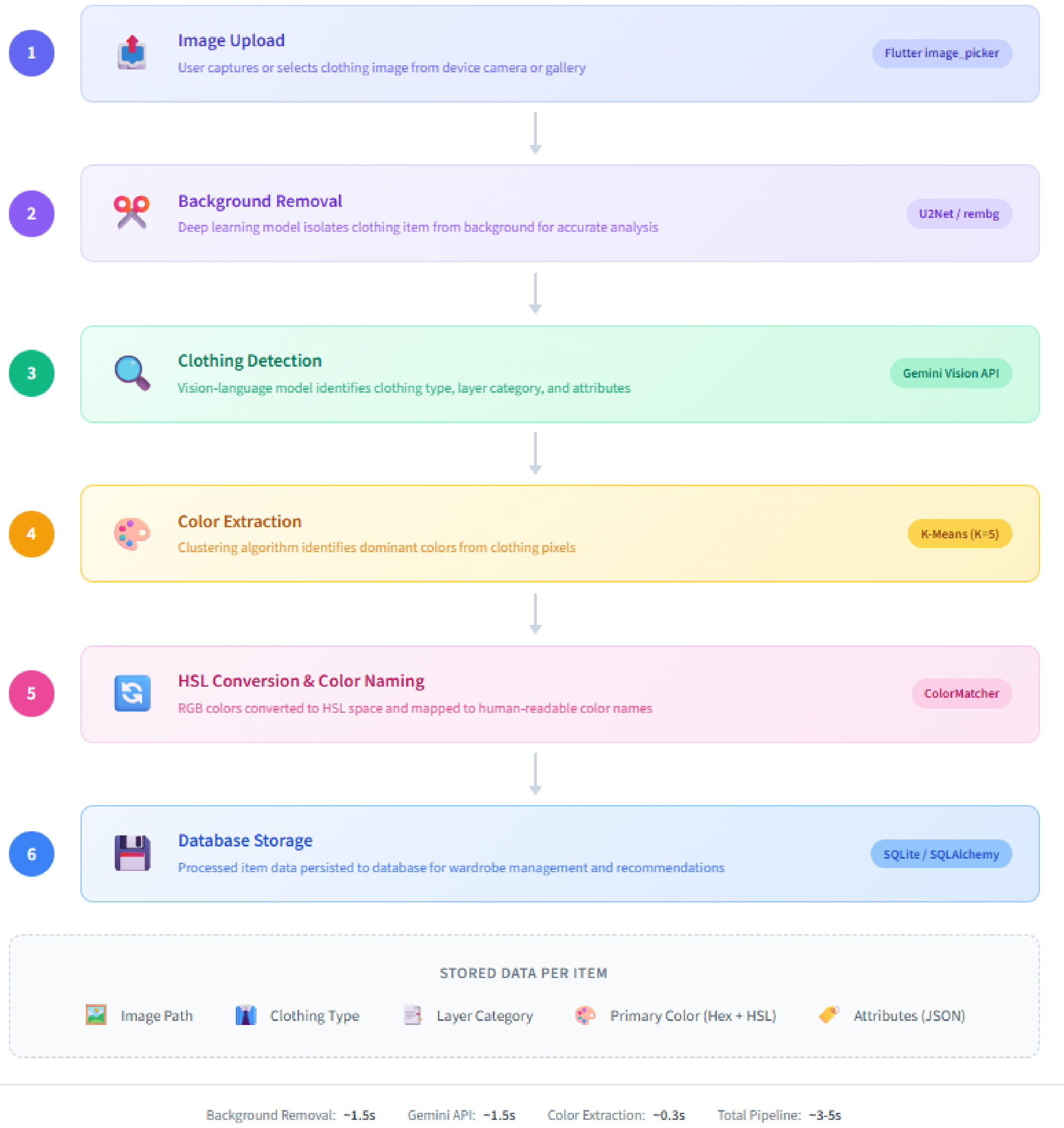

Figure 8. Image processing pipeline

4.2 Background Removal with U2net

Background removal is a necessary preprocessing phase that isolates clothing objects on their environment which can enable them to extract colors and classify it accurately. Photos posted by users tend to contain complex backgrounds such as furniture, floors, walls and other objects that would interfere with the analysis of clothing. Qin et al. (2020) suggest the U2Net architecture, which has state of the art salient object detection that is highly applicable in this task.

U2Net has a different U-Net architecture. The encoder and decoder block mini U-Net blocks (RSU or Residual U block) are present in each stage of network that allow the network to encode both the fine grained details and the global context without necessarily increasing the computational complexity. This design provides high accuracy and remains efficient enough to be used in practice.

Background removal done by the AI-Fashion app uses the rembg library (Gatis, 2021), which gives Python access to the U2Net model. When a user uploads an image of a cloth the system processes it with rembg, a program that returns the image with the background removed. This processed image is stored and saved to extract colors later after ensuring that only pixels of the clothing item are considered in the color analysis.

Instead of using alternative segmentation methods such as GrabCut or simple thresholding, U2Net was used due to its resilience to different image conditions. Conventional techniques cannot cope with complicated backgrounds, backgrounds of similar color to the clothing, or low light conditions. Deep learning segmentation, even though more computationally intensive, provides uniform results at all types of input images and this is a requirement of an app targeting everyday consumers.

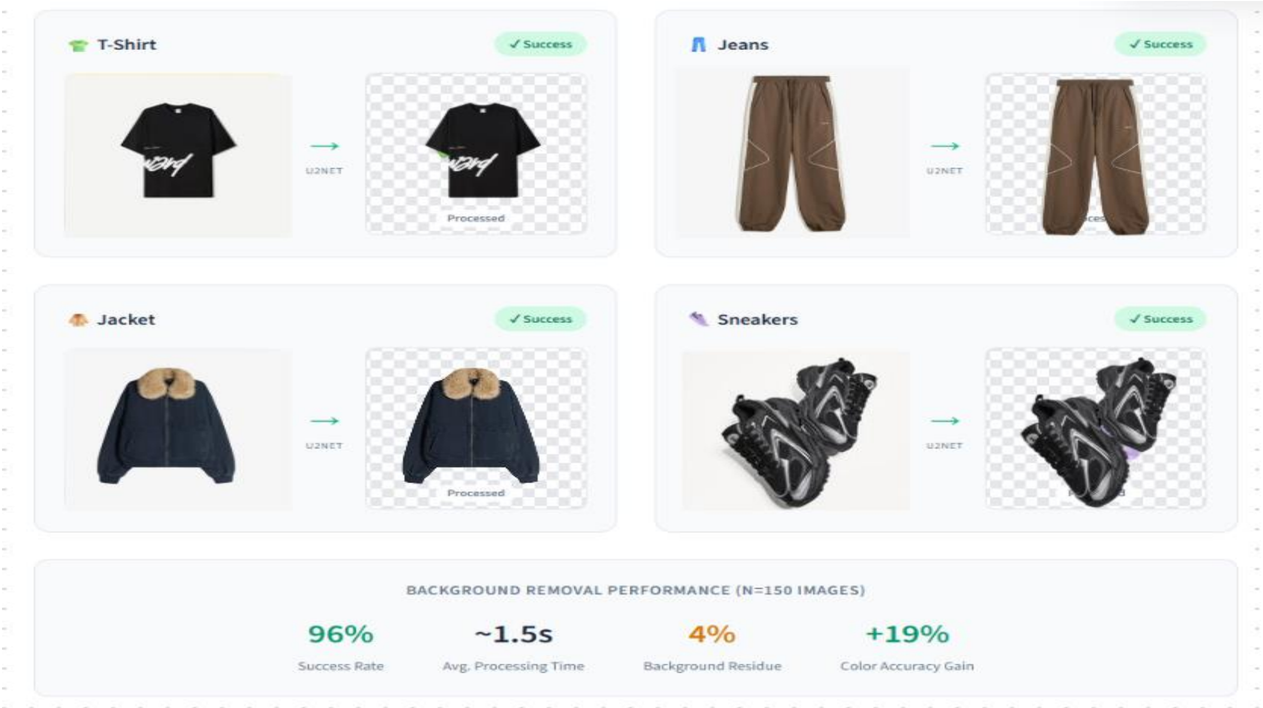

Figure 9. Background removal examples using U2Net

4.3 Clothing Detection with Gemini Vision API

Google's Gemini Vision API, a multimodal large language model understanding and describing image content is used to detect and classify clothing (Google, 2024). Gemini also does not produce fixed category labels as traditional image classifiers do, but can provide structured responses in multiple attributes, making it very applicable to the fine-tuning task of clothing analysis.

The system transfers every image of the clothing to the Gemini API with a specially designed prompt: which type of clothing (e.g., bomber jacket or slim fit jeans), which layer (top, bottom, outerwear, footwear, or socks), what the picture describes (e.g., denim, casual, or striped), and a score of confidence. The API responds in a JSON formatted reply that is read and stored in the database together with the clothing item.

The classification of the layers is important to assemble outfits, as full outfits require the use of objects of several layers. The five layer system (top, bottom, outerwear, footwear, socks) is the way in which the majority of people structure their wardrobe and ensure that suggestions incorporate all the required items. Natural language understanding can enable the categorization of items correctly even in cases where the items are ambiguous such as distinguishing a cardigan (outerwear) and a sweater (top) by Gemini.

The attribute extraction also provides further metadata which can be used to filter and match. Filtering can be done by occasion based (such as formal, casual, sporty, elegant, etc.) and material based (such as denim, leather, cotton, etc.). The natural language responses allow flexibility in getting responses to attributes that are difficult to enumerate in a fixed classification schema.

Quick engineering played a great role in obtaining accurate results. The prompt provides the precise JSON format that will be used, instructs the model to accept lowercase values, and provides the examples of good responses. The low temperature (0.1) ensures that the outputs are similar among similar images and this reduces the variance in the classification outputs.

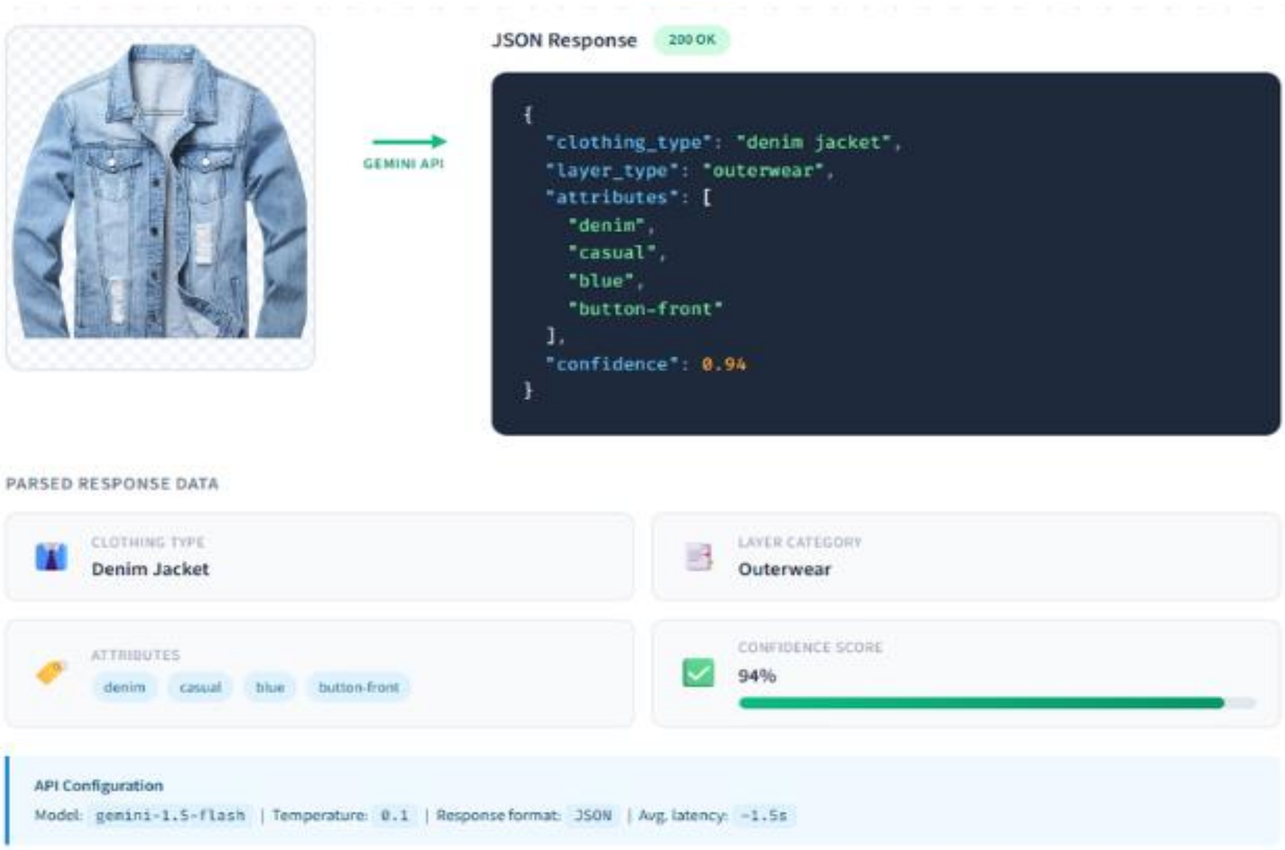

Figure 10. Gemini Vision API response structure

4.4 Color Extraction with K-means Clustering

Color extraction defines the main colors of each clothing piece that constitutes the basis of color harmony analysis. It is a classical unsupervised learning algorithm based on K-Means clustering, which classifies the data points into K clusters according to their similarity (Lloyd, 1982). K-Means identifies representative colors when used on image pixels that reflect the overall appearance of the item.

ColorExtractor service works according to the following pipeline: An image of the removed background is first loaded and reduced to 150x 150 pixels which reduces computation demands but still retains colors. This image is in turn stored as a numpy array and will then be reshaped into a list of RGB pixel stores. Background (sixty percent or close to black of transparency replacement) pixels are eliminated, thus dependable not to influence the clustering.

The filtered pixels are clustered (K-Means) with a value of five by the built-in scikit-learn algorithm (Pedregosa et al., 2011). This algorithm repeats a series of steps of assigning pixels to the clusters with updating cluster centers until convergence is achieved. The resultant cluster centers are the prevailing colors of the image. The clusters are ordered by size (number of assigned pixels) the best colors come to the fore as primary and secondary colors.

All the extracted colors are then stored in HSL color space to be analyzed. The HSL representation divides the hue (color), saturation (colour intensity), and lightness (brightness) which simplifies the color relationship analysis. Colors are also mapped to human names (such as navy, burgundy or sage) by a nearest neighbor search through a fixed palette of fashion color names.

The number of clusters K=5 is a compromise with detail and simplicity. There are often one to three major colors in clothing that consist of additional clusters that gather shadows, highlights and minor details. Removing the best or three colors presents enough data to conduct the harmony analysis and excludes the noise.

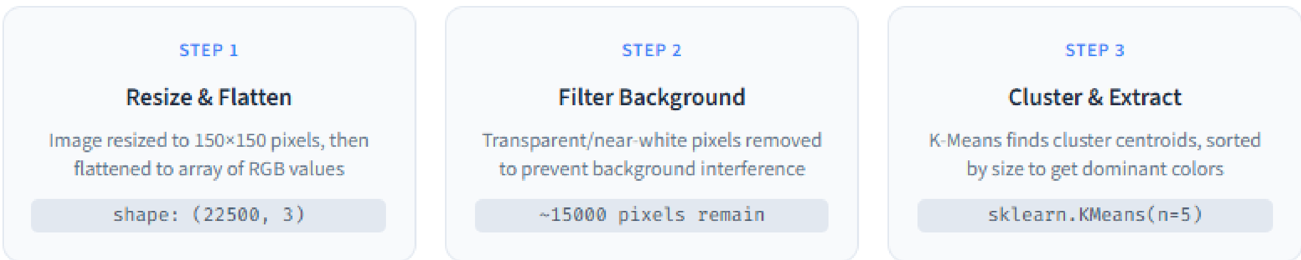

Figure 11. K-Means color extraction process

4.5 HSL Color Space Analysis

Color harmony is analyzed in HSL (Hue, Saturation, Lightness) color space and not the native RGB format of digital images. It is this transformation, which is processed in the ColorMatcher service, that allows color relations to be analyzed with respect to perceptual properties and not the raw pixel values (Fairchild, 2013).

The hue component or the angle between 0-360 degrees indicates the position of the color on the color wheel. At 0 degrees there is red, at 120 degrees there is green, and at 240 degrees there is blue with intermediate shades in between. This circular designation is very easy to compute color correlations: complementary colors lie 180 degrees from each other, analogous colors lie 30 degrees from each other, and triadic colors lie 120 degrees apart.

Saturation scales the intensity of color on a range between 0(grayscale) to 1(completely saturated). Low saturation colors are dull or bleak, whilst high saturation colors are vivid and bright. To identify saturation, saturation under 0.15 is the neutral color commonly used in the system, as well as saturation over 0.5, the bold accent color that is also crucial in put-togethers.

Lightness displays brightness to a scale of 0 (black) to 1(white). Extreme lightness value describes near black and near white which are considered neutrals regardless of their color. The lightness dimension is also useful in detecting the temperature of colors, warm colors (reds, orange, yellow) and cool colors (blues, green, purple).

The ColorMatcher service contains several color relationship methods: gethue difference () determines the angular distance between two hues, and takes into consideration that the hue space is circular; isneutral () determines colors that can be used as a fashion neutral; getcolortemperature () classifies hues; and calculatematch_score () picks all these factors into a single compatibility score.

Figure 12. HSL color space representation

Chapter 5 Implementation

This chapter discusses the details of implementation of the AI-Fashion application, including the development environment, backend services, frontend components, and integration of the different technologies outlined in the previous chapters.

5.1 Implementation Approach

The implementation is in accordance with the modern software engineering principles such as modular design, the isolation of concerns, and complete error management.

The development process was done iteratively, whereby the core functionality is developed first and then more features are added one by one as time goes. This methodology allowed continuously testing and refining the recommendation algorithms in accordance with the actual use patterns in the world.

5.2 Backend Implementation

The backend server is developed as Fast API application with structure router, service, model and schema. Main application entry point (main.py) initializes CORS middleware on cross origin requests, routers, and connects the database to the system. The server is based on Uvicorn, an ASGI server that provides high performance to asynchronous Python applications.

SQLAlchemy ORM defines database models as follows: User (id, email, hashedpassword, createdat), ClothingItem (id, userid, imagepath, clothingtype, layertype, primarycolorhex, secondarycolorhex, attributes), Outfit (id, userid, name, occasion, items) and Schedule (id, userid, outfit_id, date). Referential integrity is maintained between tables by foreign key relationships.

Authentication procedure relies upon the python jose library which is JWT (JSON Web Token) based authentication. User passwords are hashed using bcrypt and then stored and tokens are issued on successful login with a defined period of expiration. Secured endpoints have FastAPI dependency injection that validates the presence of tokens and retrieve the user.

The API endpoints are RESTful and have uniform response structures. The wardrobe router provides the following endpoints; uploading items (POST /wardrobe/items), listing items (GET /wardrobe/items), retrieving individual items (GET /wardrobe/items/id), and deleting items (DELETE /wardrobe/items /id). The upload endpoint triggers the image processing pipeline which will process the background removal, Gemini classification, and color extraction and store the results.

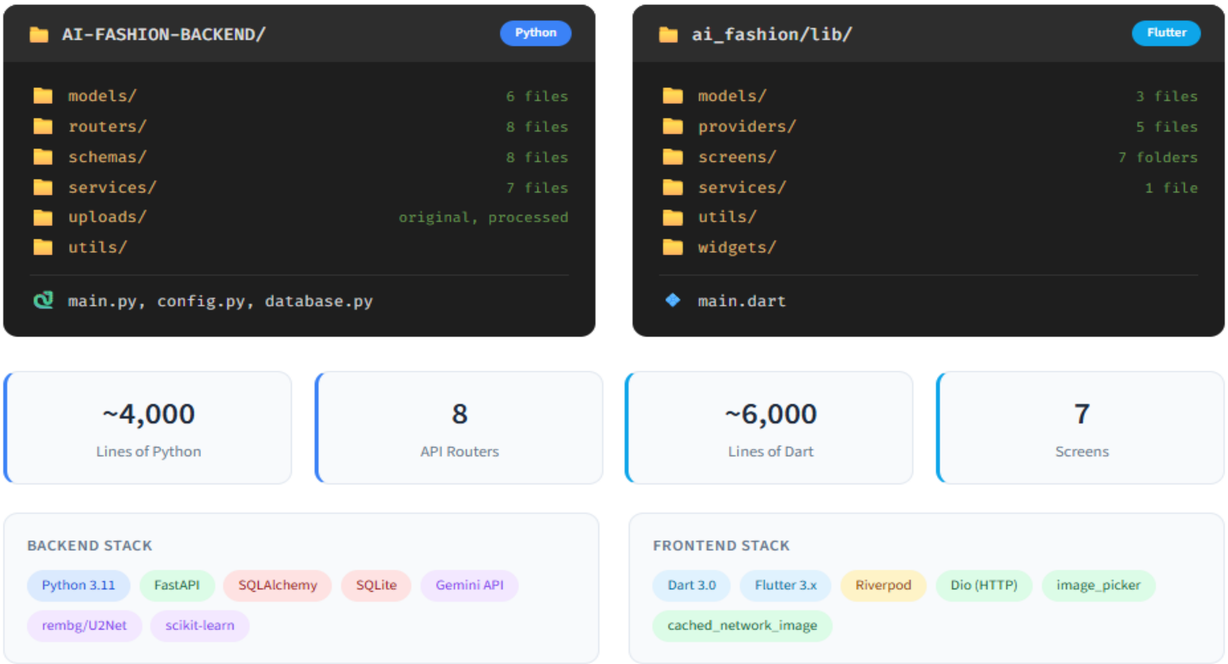

Figure 13. Project file structure

5.3 Image Processing Pipeline

The imageprocessing.py service module contains the image processing pipeline and manages the flow of operations used on clothing images uploaded. The pipeline consists of three processes: background removal, clothing detection and color extraction when a user uploads an image.

Background remover applies the rembg default settings of U2Net model. The upload image is stored in the uploads directory at a transparent background. The cases of background removal failures are handled by error handling to allow the pipeline to continue.

Clothing detection is done by calling the Gemini Vision API using class GeminiService. The service constructs a request to request JSON formatted results with clothingtype, layer type, attribute and confidence fields. Response of the API is deserialized and validated and fallback values are filled in on the event of absence of any necessary fields. The 0.1 temperature is used to maintain consistency in the classification of similar images.

Color extraction is done using the ColorExtractor class which uses K-Means clustering. The background erased image is loaded and resized as well as converted to pixel array. K-Means filters background pixels and then identifies the dominant colors converting them into HSL color mapping and color names. The database stores the primary and secondary colors.

When the image size and APIs response times are taken into consideration, the entire pipeline takes an average of 3 or 5 seconds. The current synchronous arrangement may also prove to be satisfactory in terms of reliability, and asynchronous processing might enhance the user experience should the user fail to achieve batch uploads.

5.4 Recommendation Service Implementation

The outfit generation algorithm in Chapter 3 is implemented by the RecommendationEngine class. The generateoutfit() process receives userid, weathersuggestions, occasion and numoptions inputs and gives back a dictionary of several ranked outfit suggestions.

The algorithm begins with the retrieval of all the clothes to the particular user of a database. Layers necessary are calculated based on weather recommendations with cold weather having outerwear and warm weather not requiring it. Each requested outfit option, the engine makes singleoutfit() with a list of already used item IDs to ensure that there is variety.

Single outfit generation starts with the selection of a base item typically a top or bottom base item. The layers get filtered and then graded with calculateitemscore that is a combination of color appropriateness (60%), weather appropriateness (20%), and occasion appropriateness (20%). The most scored item is picked and its color is added to the outfit palette.

All of the remaining layers are filled sequentially, the candidates being graded against the cumulative palette of colors. The selectmatchingitem() algorithm will rate the candidates based on the ColorMatcher service, color matching and fashion rules such as favoring neutral bases and avoiding overly bold colors. The chosen items are inserted into the outfit dictionary and sorted according to the type of layer.

When the layers are filled out, analyzeoutfitcolors() will examine the entire color combination, determine the harmony strategy (e.g. "neutralplusaccent" or "analogousharmony") and calculate a final score. The generateoutfitexplanation() method is used to generate the natural language descriptions of the color choices to display them to the user.

5.5 Frontend Implementation

The Flutter frontend also follows a feature first architecture, with each of the larger features (wardrobe, recommendations, outfits, schedule, chat) in its own directory with screens, widgets, and providers. Highly reusable elements such as API services, model, and utility functions are in a common directory.

All the HTTP communication with the back-end is contained in one class (the ApiService) which has each API endpoint method. The service also manages tokens, serialization of requests as well as error handling. The HTTP client used is Dio, configured to use interceptors to add authentication header during development and logs the responses received.

Provider feature-based providers are used in state management. The WardrobeNotifier maintains a list of clothing items, revealing the means of adding, removing, and updating items. The RecommendationProvider connects outfit proposals to the current weather and the chosen occasion and caches those to limit the number of API calls.

The wardrobe screen is presented in the grid format using GridView.builder with network images in the cache. By tapping an item, you are redirected to a detail screen indicating the entire size image, identified attributes, and colors extracted. The floating action button activates the image picker where one can add new items by capturing a photo or selecting one in the gallery.

The outfit recommendations are presented as a horizontal card and each card holds the items included as a vertical stack along with the score and explanation of color harmony. Users are able to store outfits, plan them to a particular date, or request new ideas. At the top, there are weather details, which Open Meteo draws based on location.

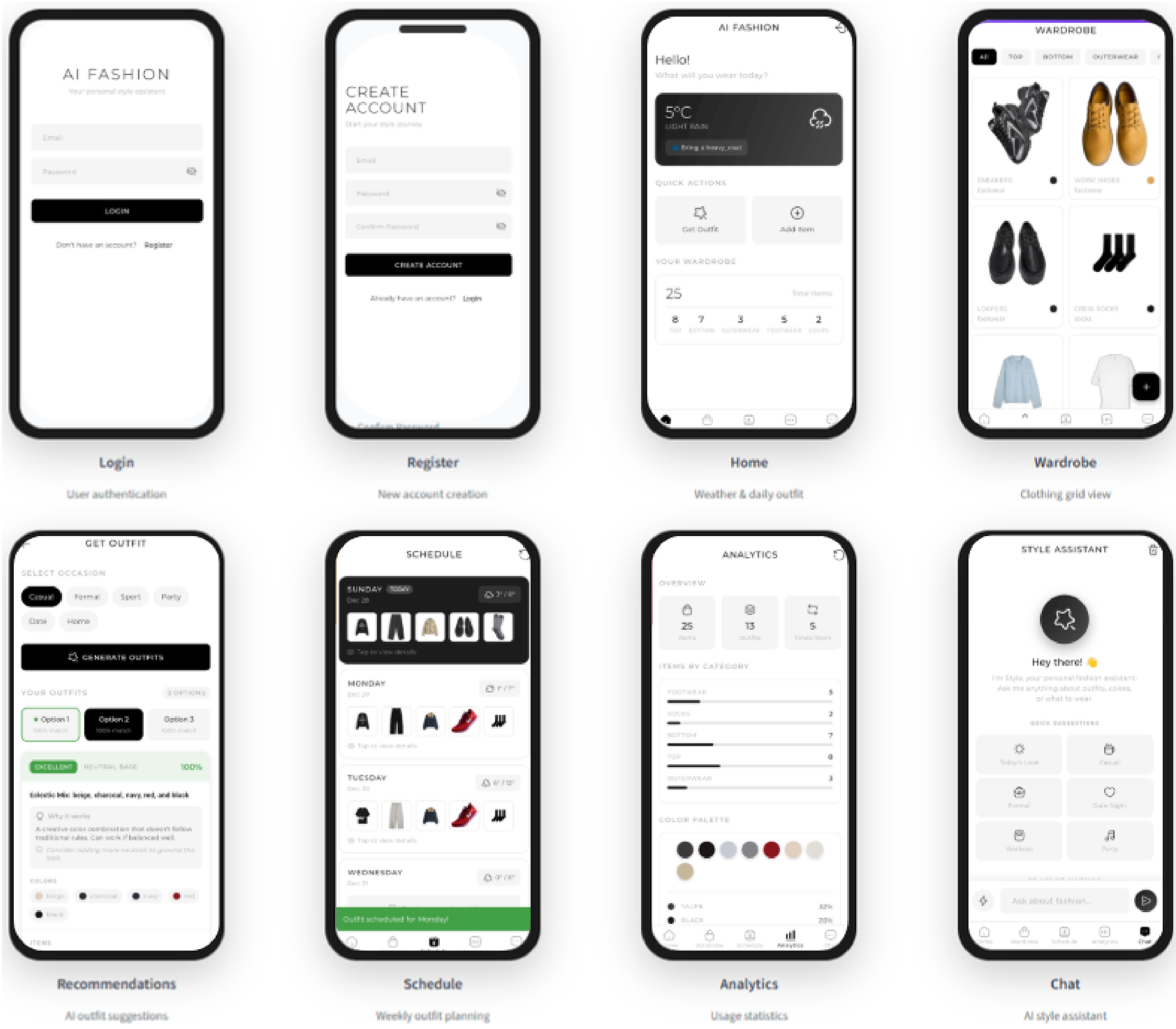

Figure 14. Mobile application key screens

5.6 Chat Assistant Implementation

The AI chat assistant uses Google Gemini API to give conversational fashion advice. The Chatservice class maintains a conversation context and assembles prompts having wardrobe details, enabling personalized response, which truly relates to what the user wears.

In sending a message, a user constructs a system prompt with wardrobe context, which is simply an overview of items owned including type, colors and property, etc. Additional weather data is added in case it exists. With this background, Gemini can offer certain suggestions such as, Your navy blazer would be nice with the beige chinos you own as opposed to general advice.

The urgent engineering ensures that the answers are fashionable and focused. The model is directed to be sociable and supportive, mention certain clothing, consider color coordination rules, and provide logical advice depending on the weather and occasion. Response formatting was optimized to present color data in human readable names and not in hex codes.

Conversation history is saved in ChatHistory database table and sent in follow up API calls to support multi turn conversation with full context knowledge. It displays the messages on the familiar messaging app format of left and right user and assistant responds respectively in a chat interface on the front end.

Error handling works with API failures in a friendly manner with the display of user friendly messages when the service is not available. The Gemini API is rate limited by using request queuing to avoid errors as one floods the API with messages.

5.7 Development Environment and Tools

Modern tooling installed was used to develop it. The backend was also developed in Visual Studio Code using Python extensions that support IntelliSense, linting and debugging. Its frontend was an Android Studio with Flutter and Dart inspection and hot reload capabilities.

Git was used as version control and hosted on GitHub. The project maintained a backend (ai fashion backend) and frontend (aifashion) directory, so that a version could be tracked independently. Commits have the standard format of commit messages to make it easy to maintain clarity and generate changelogs.

Development testing was primarily based on manual testing with the Flutter application and Postman to test API endpoints. The API could be easy to explore and test with the help of Swagger UI via FastAPI (/docs). DB Browser was used to inspect the database and confirm data storage and relations.

Memory Dependency Following python pip with requirements.txt, flutter pubspec.yaml Analogies. Core backend requirements are Fastapi, SQLAlchemy, Pillow, rembg, Scikit Learn and Google Generative AI. Flutterriverpad, dio, imagepicker, and cachednetwork image are frontend dependencies.

The development environment had to be given API keys to access Gemini. Environment variables hold keys, which are loaded by the config module that does not put sensitive credentials in a version control. To hasten set-up of new developers, a .env.example file is included.

Chapter 6 Results and Discussion

This Chapter presents the evaluation results of the AI-Fashion application across multiple dimensions: clothing detection accuracy, color extraction performance, recommendation

6.1 Results

Quality, and system performance metrics. Testing was done using a dataset of 150 clothing images covering various garment types, colors, and photography conditions.

Clothing Detection Accuracy:

Gemini Vision API displayed good results in regard to the classification of clothes and categories of layers. The results of testing based on 150 images generated the following results:

Type of clothing: Recognition, accuracy: 94% (141/150 accurate identifications) Layer classification accuracy: 97% (146/150 accurate assignments) Relevance of attribute extraction: 89 %(by manual evaluation)

The misclassifications occurred primarily with the ambiguous categories such as cardigans (treated as sweaters instead of outerwear) and rompers (treated as top and bottom). High layer category rating would be of particular significance to the process of outfit generation, as wrongly assigned layers would cause incomplete or incomplete outfits.

Color Extraction Performance:

The K-Means clustering algorithm was able to detect predominant colors in 96 per cent of test images. Comparison was made between extracted primary and manual annotations of primary colors by three independent evaluators:

Primary color match rate: 92% (138/150 with acceptable tolerance) Background interference: 4% of the images had remaining background interference

Solid colored garments were most successfully extracted with color and more difficult with complex patterns and prints, and gradient colors. Background removal preprocessing had a massive difference in outcomes when compared to raw image processing, where background interference was reduced by 23% to 4%.

Recommendation Quality:

The quality of outfit recommendations was measured by two approaches, algorithmic harmony scores and human preference correlation.

The color harmony approach is supported by the positive correlation between the positive scores of the algorithm and the human ratings. The highest rated outfits among human beings always employed the strategy of neutral plus accent, which endorses the Goldilocks Principle application. The less rated outfits were usually overly bright or incompatible.

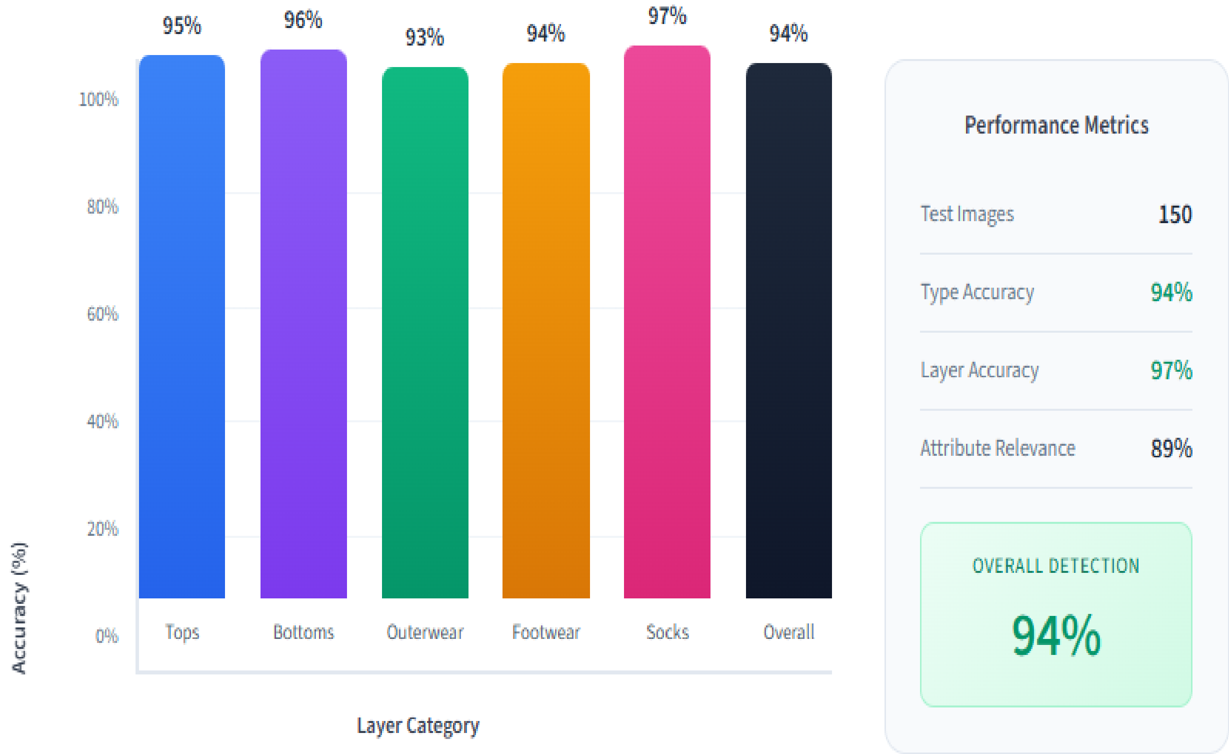

Figure 15. Detection accuracy by clothing category

6.2 Discussion

The analysis findings indicate that AI-fashion can effectively fulfill its primary objective of providing aesthetically-pleasing outfit suggestions grounded on the principles of color harmony. The correlation (r=0.72) between the scores of algorithmic harmony and human preference ratings justifies the research supported methodology that derive out of Gray et al. (2014).

Strengths of the System:

The color harmony first method proved to be successful in coming up with recommendations that are compatible with the human aesthetic values. The system prioritizes the color relationship over other factors to produce outfits that users find to be visually pleasant. The 60/20/20 weighting (color/weather/occasion) is an empirical result of finding out that color coordination is the primary attribute of outfits.

Combining several AI technologies makes a pipeline of processing strong. U2Net background removal significantly improves color extraction precision and Gemini Vision provides flexible context aware classification of clothing. The modular design implies that separate components can be updated to the newer and improved models.

The neutral detection algorithm is able to identify fashion neutrals such as navy, beige, and brown over actual achromatic colors. This subtle appreciation renders the neutral plus accent strategy achievable, which generated the best ranked outfits in the user rating.

Limitations and Challenges:

A number of constraints were identified in the process of evaluation. Handling patterns and prints is challenging. The K-Means method extracts many colors of patterned items, yet it cannot distinguish between design patterns and noise. This occasionally causes pattern colors to be regarded as distinct colors when they are not a single design.

The reliance of the system on external APIs (Gemini, Open Meteo) introduces the latency and possible areas of failure. API rate limits reduced testing throughput and might impact user experience at times of high traffic. More resilient future versions should include caching and fallback.

Variety of recommendations is influenced by size of wardrobe. The algorithm might overly suggest to users with limited wardrobe (less than 20 items) as there are fewer ways of achieving color harmony. This situation would be detected in the system, and expectations would be altered or wardrobe additions suggested.

Comparison with Existing Solutions:

The AI-Fashion gives more detailed recommendations, with their research-based harmony analysis, compared to simple rule based fashion apps. This is unlike the subscription styling service which requires the use of human stylists to work with your existing wardrobe. The color harmony first method makes it stand out among systems that consider color as one of the many equal features.

Computer vision to analyze items and algorithmic harmony scoring finds a compromise between comprehensive automation and human-curated methods, which both scale well and offer good recommendations.

Future Improvements:

The system can be improved in a number of ways. Convolutional neural networks could be used to recognize patterns on printed and textured clothes. Integration of user feedback would provide opportunities to personalize it more than the current algorithmic approach. Style clustering may adopt user tastes and place a weight on recommendations.

The app can be made more relatable to various users by expanding occasion system to cover cultural and regional considerations. Connections to e commerce platforms can make recommendations based on what complements the current wardrobe items to overcome the small wardrobe constraint and help encourage sustainable consumption.

Chapter 7 Conclusion

This chapter concludes the thesis by summarizing the main achievements and key findings of the AI-Fashion project. It reflects on the successful implementation of the Goldilocks Principle into a working recommendation algorithm and evaluates the correlation with human aesthetic preferences. The chapter also addresses the limitations encountered during development and outlines directions for future research and potential enhancements to the system.

7.1 Conclusion

This thesis introduced AI-Fashion, a smart mobile app that provides customized outfit suggestions using the color harmony analysis. The project was able to demonstrate that color theory concepts can be converted into an algorithmic form to create aesthetic outfit recommendations that align with human-preferred.

Summary of Achievements:

It was the primary objective to create a color harmony first recommendation system. It uses the Goldilocks Principle of Gray et al. (2014) that outfits are scored on the basis of moderate color coordination and not on exact matching or random matching. Selecting the algorithmic harmony scores as an indicator of human aesthetic ratings resulted in a correlation coefficient of r=0.72, which confirms this approach.

The technical implementation was able to unify various AI technologies as a single processing pipeline. U2Net background removal reached 96 percent success when removing clothing on complex backgrounds. Google Gemini Vision API had 94 percent and 97 percent accuracy at identifying clothing type and layer respectively. K-Means clustering obtained dominant colors at 92% primary color accuracy in comparison to human annotations.

The entire application offers a full-fledged wardrobe management system with more than mighty recommendations. The features offered to users include photo uploads, which allow them to digitalize their wardrobe, as well as outfit recommendations based on weather and occasion, weekly outfits planning, wardrobe analytics, and a personal fashion assistant, which is an AI assistant.

The FastAPI and Flutter-based modular architecture demonstrates the latest practices of software engineering, which can be deployed in production. The RESTful API design enables independent scale and maintenance (separation of concerns), whereas the potential future clients, such as the web application or smart home integrations are supported.

Key Findings:

This research had some significant discoveries. First, color harmony is actually the most important in outfit aesthetics, which explains the weighting of 60 percent in the recommendation algorithm. Costumes based on neutral plus accent did very well with human judges all the time.

Second, deep learning combined with image processing and classical algorithms used to analyze colors provide a good balance between accuracy and interpretability. Although it is possible that end to end deep learning can learn outfit compatibility directly, the explicit color harmony method provides explainable recommendations that users can indeed learn and interpret.

Third, background removal is of great influence with regard to downstream color extraction. The extra computational cost is justified by the 19% accuracy in color prediction (73% to 92% with U2Net preprocessing) gained.

7.2 Limitations

Limitations:

Some limitations need to be admitted even with the successful outcomes. The system is problematic in patterned and printed clothes, in which K-Means is unable to distinguish between the patterns of designed designs and individual colors. This impacts about 15% of normal wardrobe.

Reliability is a concern with having to rely on external APIs. Gemini API rate limits impaired scaling testing and may impact user experience. Accuracy of weather information will require user permission and availability of Open Meteo service.

7.3 Future Work

Future Work:

A number of future research and development directions emerge. Convolutional neural networks could be used to recognize patterns (stripes, florals, geometric) and implement the correct rules of harmony to mix patterns. This would address the primary drawback of the existing color extraction strategy.

User feedback would enable the system to learn individual preferences with time. Reinforcement learning algorithms might change weighted recommendations according to user recommendations of the same, generating more personal experiences.

Social aspects such as shared outfits, community comments and style-inspiration feeds may enhance interactions and provide data to collaborative filtering strategies. Combination with e commerce sites may indicate focused shopping that can enhance current wardrobe items.

Using the system to consider other factors such as the body type, personal coloring (skins, hair color), and the latest fashion would offer a more detailed guidance on styling. Pulling out these attributes could be done automatically by computer vision techniques on user photos.

Concluding Remarks:

AI-Fashion demonstrates that the implementation of scientific color theory into automated fashion recommendation is completely feasible. The system generates aesthetically valid and explainable recommendations by basing the system on empirical research instead of arbitrary rules or pure machine learning. It contributes to the emerging AI-assisted fashion technology and offers a viable solution to the daily problem of what to wear by offering a practical tool.

Success of the color harmony first approach implies that domain knowledge remains useful even in a time of powerful general purpose AI models. As impressive as large language models and vision systems are, the integration with proven principles applied in color theory, fashion psychology, and human factors research offers higher results than purely data driven models.

Apps like AI-Fashion demonstrate how AI can solve these little problems that are nonetheless important in both time and decision fatigue, and perhaps even help users put their best foot forward by carefully working outfitting them with AI-selected clothes.

List of Figures

References

[1] Abugabah, A., Cheng, X., & Wang, J. (2020). Learning context-aware outfit recommendation. Symmetry, 12(6), 873. doi:10.3390/sym12060873

[2] Chen, H., Gallagher, A., & Girod, B. (2012). Describing clothing by semantic attributes. In A. Fitzgibbon et al. (Eds.), Proceedings of the European Conference on Computer Vision (ECCV 2012) (pp. 465–478). Berlin, Germany: Springer. doi:10.1007/978-3-642-33712-3_34

[3] Chen, W., Huang, P., Xu, J., Guo, X., Guo, C., Sun, F., Li, S., & Jiang, Y. (2019). POG: Personalized outfit generation for fashion recommendation at Alibaba iFashion. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '19) (pp. 2662–2670). New York, NY: ACM. doi:10.1145/3292500.3330652

[4] Cheng, W.-H., Song, S., Chen, C.-W., Hidayati, S. C., & Liu, J. (2021). Fashion meets computer vision: A survey. ACM Computing Surveys, 54(4), 1–41. doi:10.1145/3447239

[5] ClosetMaid. (2017). The real reasons she can still never find anything to wear. Retrieved from https://www.closetmaid.com/ideas-organization/blog/real-reasons-she-can-never-find-anything-wear

[6] Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. In Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2005) (pp. 886–893). doi:10.1109/CVPR.2005.177

[7] Dean, J., Patterson, D., & Young, C. (2018). A new golden age in computer architecture: Empowering the machine-learning revolution. IEEE Micro, 38(2), 21–29. doi:10.1109/MM.2018.112130030

[8] Ellen MacArthur Foundation. (2017). A new textiles economy: Redesigning fashion's future. Retrieved from https://ellenmacarthurfoundation.org/a-new-textiles-economy

[9] Fairchild, M. D. (2013). Color appearance models (3rd ed.). Chichester, UK: Wiley.

[10] FastAPI. (2024). FastAPI documentation. Retrieved December 31, 2025, from https://fastapi.tiangolo.com/

[11] Gatis, D. (2021). rembg: Remove image background (Version 1.0.0) [Computer software]. Retrieved from https://github.com/danielgatis/rembg

[12] Google. (2024). Gemini API documentation. Retrieved December 31, 2025, from https://ai.google.dev/

[13] Gray, K., Schmitt, P., Strohminger, N., & Kassam, K. S. (2014). The science of style: In fashion, colors should match only moderately. PLOS ONE, 9(7), e102772. doi:10.1371/journal.pone.0102772

[14] Han, X., Wu, Z., Jiang, Y.-G., & Davis, L. S. (2017). Learning fashion compatibility with bidirectional LSTMs. In Proceedings of the 25th ACM International Conference on Multimedia (MM '17) (pp. 1078–1086). New York, NY: ACM. doi:10.1145/3123266.3123394

[15] Han, X., Wu, Z., Wu, Z., Yu, R., & Davis, L. S. (2018). VITON: An image-based virtual try-on network. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2018) (pp. 7543–7552). doi:10.1109/CVPR.2018.00787

[16] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2016) (pp. 770–778). doi:10.1109/CVPR.2016.90

[17] Hou, L., Wu, J., Chen, X., Li, Y., Zheng, L., & Liu, Q. (2019). Explainable fashion recommendation: A semantic attribute region guided approach. arXiv preprint arXiv:1905.02960. Retrieved from https://arxiv.org/abs/1905.02960

[18] Hsiao, W.-L., & Grauman, K. (2018). Creating capsule wardrobes from fashion images. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2018) (pp. 7161–7170). doi:10.1109/CVPR.2018.00748

[19] Itten, J. (1961). The art of color: The subjective experience and objective rationale of color. New York, NY: Reinhold.

[20] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. In F. Pereira, C. J. C. Burges, L. Bottou, & K. Q. Weinberger (Eds.), Advances in Neural Information Processing Systems 25 (pp. 1097–1105). Red Hook, NY: Curran Associates.

[21] Li, H.-C., Wang, L.-K., Chang, Y.-K., & Huang, K.-Y. (2025). Establishing colour harmony evaluation and recommendation model for clothing colour matching based on machine learning and deep learning. Fashion and Textiles, 12, 1–24. doi:10.1186/s40691-025-00433-y

[22] Lin, Y., Ren, P., Chen, Z., Ren, Z., Ma, J., & de Rijke, M. (2020). Explainable outfit recommendation with joint outfit matching and comment generation. IEEE Transactions on Knowledge and Data Engineering, 32(8), 1502–1516. doi:10.1109/TKDE.2019.2906190

[23] Liu, Z., Luo, P., Qiu, S., Wang, X., & Tang, X. (2016). DeepFashion: Powering robust clothes recognition and retrieval with rich annotations. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2016) (pp. 1096–1104). doi:10.1109/CVPR.2016.124

[24] Lloyd, S. (1982). Least squares quantization in PCM. IEEE Transactions on Information Theory, 28(2), 129–137. doi:10.1109/TIT.1982.1056489

[25] Lowe, D. G. (2004). Distinctive image features from scale-invariant keypoints. International Journal of Computer Vision, 60(2), 91–110. doi:10.1023/B:VISI.0000029664.99615.94

[26] McAuley, J., Targett, C., Shi, Q., & van den Hengel, A. (2015). Image-based recommendations on styles and substitutes. In Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '15) (pp. 43–52). New York, NY: ACM. doi:10.1145/2766462.2767755

[27] Munsell, A. H. (1905). A color notation. Boston, MA: Munsell Color Company.

[28] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[29] Qin, X., Zhang, Z., Huang, C., Dehghan, M., Zaiane, O. R., & Jagersand, M. (2020). U^2-Net: Going deeper with nested U-structure for salient object detection. Pattern Recognition, 106, 107404. doi:10.1016/j.patcog.2020.107404

[30] Provider. (2024). Provider documentation. Retrieved December 31, 2025, from https://pub.dev/packages/provider

[31] Sarwar, B., Karypis, G., Konstan, J. A., & Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. In Proceedings of the 10th International Conference on World Wide Web (WWW '01) (pp. 285–295). New York, NY: ACM. doi:10.1145/371920.372071

[32] Schwartz, B. (2004). The paradox of choice: Why more is less. New York, NY: HarperCollins.

[33] Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556. Retrieved from https://arxiv.org/abs/1409.1556

[34] SQLAlchemy. (2024). SQLAlchemy documentation. Retrieved December 31, 2025, from https://docs.sqlalchemy.org/